4-bit Array Multiplier: Parts A and B Report

Oct 19, 2026 · @Akshay

Team and scope

Parts A and B are complete: the RTL passes all 256 vectors, the testbench catches an injected fault, and the synthesized SKY130 netlist (61 cells, 756.976 µm²) passes the same 256 vectors at gate level.

Name	Roll number	Responsibility
Sale Sangameshwar	2024102017	Part A (RTL design and verification), Part B (logic synthesis)
Palla Rithik Reddy	2024102005	Part C (static timing analysis), Part D (physical design)

Item	Version / value
Environment	IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:2026.10`, run under WSL 2 on Windows 11
Simulator	Icarus Verilog 14.0 (devel)
Synthesis	Yosys 0.70
Process / library	SkyWater SKY130, `sky130_fd_sc_hd`, typical corner `tt_025C_1v80`
Repository	rithikreddypalla/DVD_RTL_to_GDS

Part A: RTL design

The multiplier registers its inputs and output, builds the product from 16 AND gates and 12 `full_adder` instances in three ripple-carry rows, and uses no `*` operator.

`rtl/full_adder.v` uses two `assign` statements:

```
\text{sum} = a \oplus b \oplus c_{in} \qquad c_{out} = ab + a\,c_{in} + b\,c_{in}
```

`rtl/mult_array.v` follows the Section 2 connection table exactly:

Stage	Inputs	Outputs
Partial products	pp*i*[j] = a_r[j] AND b_r[i]	16 AND gates
Row 0	pp0	p[0] = pp0[0]; r0 = {0, pp0[3], pp0[2], pp0[1]}
Row 1 (fa1_0 to fa1_3)	pp1[j], r0[j], c1[j], with c1[0] = 0	p[1] = s1[0]; r1 = {c1[4], s1[3], s1[2], s1[1]}
Row 2 (fa2_0 to fa2_3)	pp2[j], r1[j], c2[j], with c2[0] = 0	p[2] = s2[0]; r2 = {c2[4], s2[3], s2[2], s2[1]}
Row 3 (fa3_0 to fa3_3)	pp3[j], r2[j], c3[j], with c3[0] = 0	p[3] = s3[0]; p[7:4] = {c3[4], s3[3], s3[2], s3[1]}

One `always @(posedge clk or negedge rst_n)` block holds all 16 flip-flops: `a_r[3:0]` and `b_r[3:0]` (8) capture the inputs at clock edge 1, and `p[7:0]` (8) captures the array result at edge 2. An active-low `rst_n` clears all of them asynchronously. Latency is therefore 2 clock cycles, with one new product per cycle.

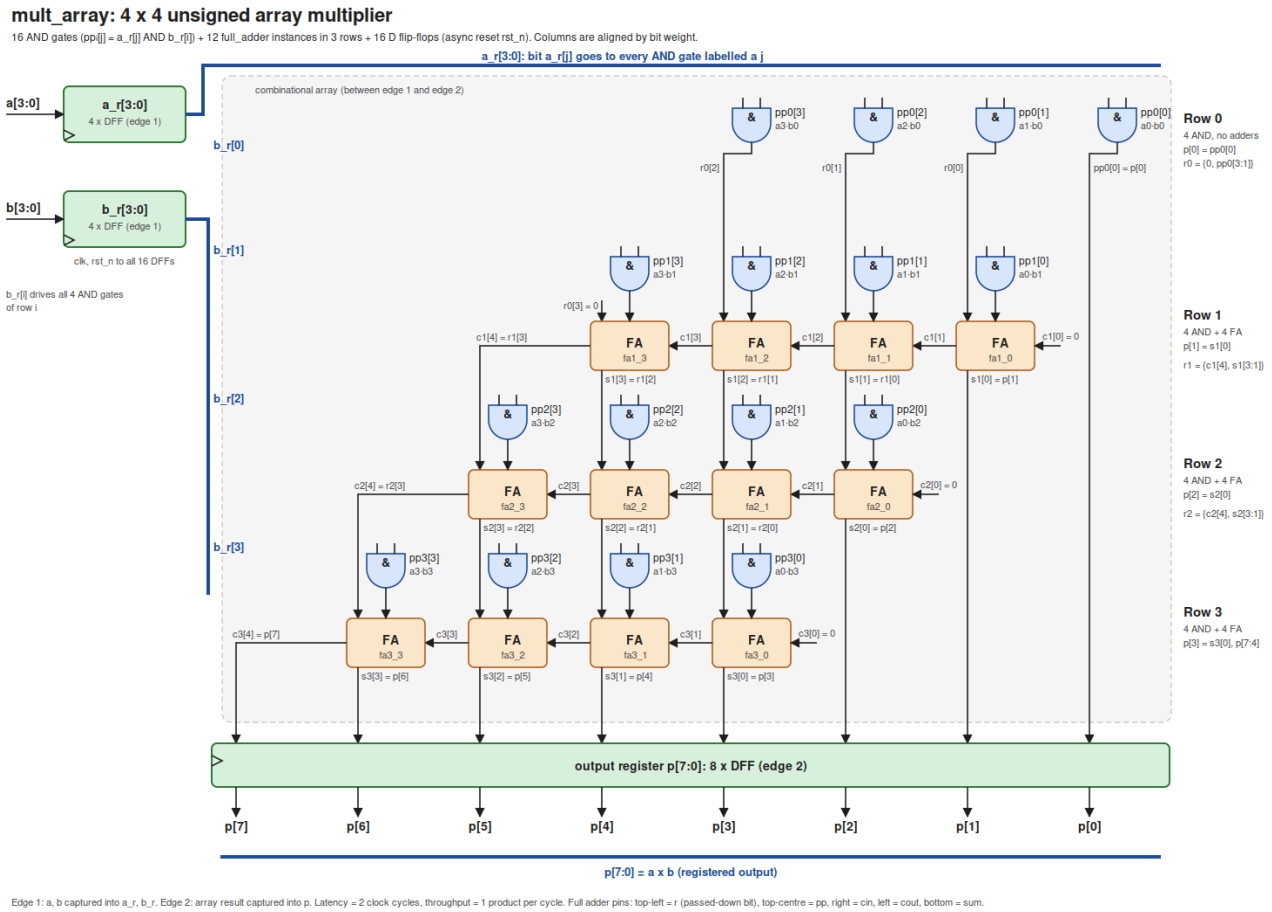


Figure 1: Block diagram of `mult_array`. Columns are aligned by bit weight, so each passed-down bit (r0, r1, r2) is a vertical wire; carries ripple right to left inside each row.

Part A: Verification

The RTL passes all 256 input combinations, and the testbench detects a deliberately broken full adder with 95 mismatches.

Testbench method (`tb/tb_mult.v`):

- 10 ns clock; `rst_n` held low for the first 2 cycles (0–20 ns), with a check that `p = 0` during reset.
- A new (a, b) pair is applied on every falling edge, for all 256 pairs (vector *i*: a = i[7:4], b = i[3:0]).
- Each vector is checked exactly 2 cycles after it is applied against the expected value `a * b` (the `*` operator appears only in the testbench). A design with latency 1 or 3 would fail.
- `!==` is used, so X or Z on `p` counts as an error. One line is printed per mismatch, then one summary line; the waveform is written to `mult.vcd`.

Run	Summary line
Correct RTL	`PASS: 256 vectors checked, 0 errors`
Fault injected: `assign sum = a ^ b;`	`FAIL: 256 vectors checked, 95 errors`
Fault removed (restored)	`PASS: 256 vectors checked, 0 errors`

The first mismatch with the fault was `a=3 b=3 expected p=9 got p=5` at 560 ns: the first vector where a full adder sees a carry-in of 1, which is exactly the term the fault removes.

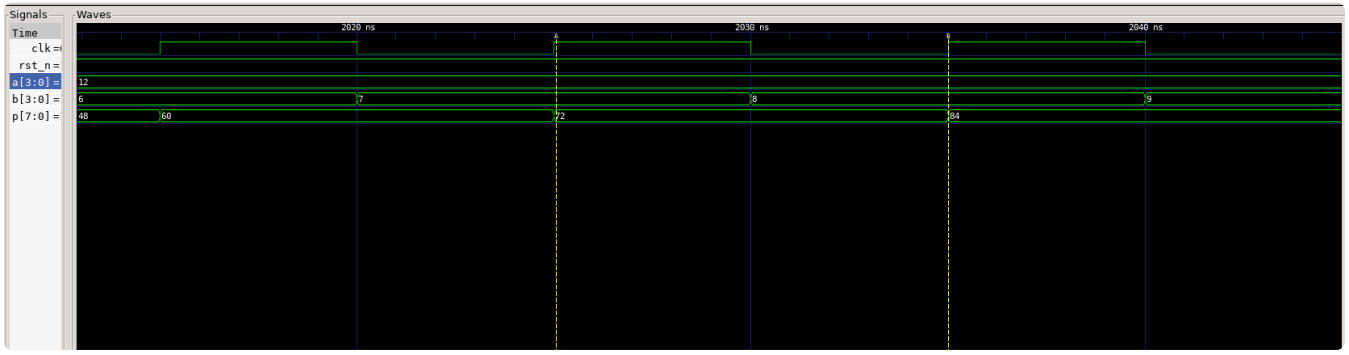


Figure 2: RTL waveform for A = 12, B = 7. Inputs change at 2020 ns (falling edge); marker A (2025 ns) is clock edge 1, where the inputs are registered; marker B (2035 ns) is edge 2, where p = 84 is registered. Latency = 2 clock cycles.

Part A: Hand-worked example

12 × 7 = 84 by hand, and the same product appears in the waveform at 2035 ns.

Roll number used: Sale Sangameshwar, 2024102017. Last digit 7, so A = 7 + 5 = 12. Since A ≠ 7, B = 7.

```
          1 1 0 0     A = 12
        x 0 1 1 1     B = 7
        ---------
          1 1 0 0     pp0 = A AND b0 (b0 = 1)
        1 1 0 0       pp1 = A AND b1 (b1 = 1), shifted 1
      1 1 0 0         pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
  0 1 0 1 0 1 0 0     P = 84
```

Traced through the array (bits written MSB first; full adders j = 0 to 3 listed as (sum, carry)):

Row	Inputs	Full adders j = 0..3	Outputs
0	pp0 = 1100	none	p[0] = 0, r0 = 0110
1	pp1 = 1100, r0 = 0110	(0,0) (1,0) (0,1) (0,1)	p[1] = 0, r1 = 1001
2	pp2 = 1100, r1 = 1001	(1,0) (0,0) (1,0) (0,1)	p[2] = 1, r2 = 1010
3	pp3 = 0000, r2 = 1010	(0,0) (1,0) (0,0) (1,0)	p[3] = 0, p[7:4] = 0101

Result: p = 0101 0100 = 84. Each row checks in decimal: 12 + 6 = 18 = 10010 (row 1), 12 + 9 = 21 = 10101 (row 2), 0 + 10 = 10 = 1010 (row 3).

In the waveform: the vector index is 16 × 12 + 7 = 199, so the testbench applies it at 30 + 10 × 199 = 2020 ns. It is registered at edge 1 (2025 ns) and p = 84 appears at edge 2 (2035 ns), as shown in Figure 2.

Part B: Logic synthesis

Yosys maps the design to 61 SKY130 cells with a total area of 756.976 μm². The gate-level netlist passes the same 256-vector testbench as the RTL.

`synth/synth.tcl` sets `$lib` to `$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib` and runs, as `yosys <command>` lines:

1. `read_verilog rtl/full_adder.v rtl/mult_array.v`
2. `synth -top mult_array -flatten` (generic synthesis; the 12 full adders are flattened into one module)

3. `dfflibmap -liberty $lib` (flip-flops mapped to SKY130 cells)
4. `abc -liberty $lib` (logic mapped to SKY130 gates)
5. `opt_clean -purge`
6. `tee -o synth/mult_array_stat.rpt stat -liberty $lib`
7. `write_verilog -noattr synth/mult_array_netlist.v`

Run with `yosys -c synth/synth.tcl` (exit code 0; the only warnings were ABC's informational notes).

Part B results table (from `synth/mult_array_stat.rpt`):

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm²)	756.976
Flip-flop area (μm²)	16 × 25.024 = 400.384
Flip-flop area (% of total)	100 × 400.384 / 756.976 = 52.89 %
The 3 most-used cell types and their counts	`dfrtp_1` × 16, `nand2_1` × 7, then `and2_0` × 4 and `maj3_1` × 4 (tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The area of one `dfrtp_1` (25.024 μm²) was confirmed in the Liberty file. `dfrtp` is a D flip-flop with an active-low asynchronous reset, matching the RTL's `rst_n` behaviour.

Gate-level simulation used the command from the assignment (the PDF wraps the netlist path across two lines; it is one path):

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v
synth/mult_array_netlist.v
vvp gls
```

Output: `PASS: 256 vectors checked, 0 errors`, so synthesis preserved the behaviour, including the 2-cycle latency and the reset.

Report questions answered from Part B

Question 1: logic cell count. Synthesis produced 61 − 16 = 45 logic cells, not 16 + 12 = 28. After `synth -flatten` the full adders no longer exist as units, and ABC re-maps the whole array as one Boolean network into library gates. A full adder needs an XOR3 for the sum and a majority function for the carry, so it costs at least two cells (`xor3_1` + `maj3_1` , or several 2-input gates). In the other direction, ABC merges AND gates into compound cells (`a21oi` , `a22oi` , `o2111ai` , `nand3` / `nand4`), and simplifies adders with a constant input: c*i*[0] = 0 in every row and r0[3] = 0. ABC also skips SKY130's multi-output full-adder cell (`fa_1` ; its log warns of "9 multi-output cells"). The count therefore reflects the optimised logic, not the drawing.

Question 3: flip-flop share of area. Flip-flops take 400.384 of 756.976 µm², or 52.89 %, although they are only 16 of 61 cells (26 %). One `dfrtp_1` is 25.024 µm², about 6.7 times a `nand2_1` (3.754 µm²), because it contains a master and a slave latch plus reset circuitry. The register count grows linearly with the port width (2N input bits + 2N output bits = 16 here), while the array logic grows with N². For a 4-bit design the N² logic is still small, so registers dominate; at 8 or 16 bits the logic share would rise.

Commands, issues and commits

All commands ran inside the IIC-OSIC-TOOLS container from `/foss/designs` (the repository root), in this order:

```
# host (WSL 2)
docker pull hpretl/iic-osic-tools:2026.10
.dev/start_container.sh

# container
cd /foss/designs
iverilog -V | head -1; yosys -V; sta -version
echo $PDK_ROOT $PDK          # showed /foss/pdks ihp-sg13g2
sak-pdk sky130A              # now /foss/pdks sky130A

iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim                      # PASS
# edit full_adder.v: assign sum = a ^ b;  -> rerun: FAIL, 95 errors
# restore assign sum = a ^ b ^ cin;      -> rerun: PASS
gtkwave mult.vcd

yosys -c synth/synth.tcl
```

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v
$CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls                      # PASS
```

Issue	Fix
Docker Desktop was not running on Windows	Used the Docker Engine running inside WSL 2 Ubuntu
The container's default PDK was `ihp-sg13g2`	Ran `sak-pdk sky130A`, as the assignment advises
Mismatch messages printed picosecond values labelled "ns"	Added `$timeformat(-9, 0, " ns", 0)` to the testbench
`.dev/*.sh` checked out with Windows CRLF line endings, so bash rejected them	Converted to LF
WSL has about 3 GB RAM, below the recommended 8 GB	Enough for Parts A and B; to be raised in `.wslconfig` before the LibreLane run

Commit	Contents
`dbb2398` Part B: Yosys synthesis	`synth/synth.tcl`, netlist, stat report, `README.md`
`d2e4803` Part A: RTL and testbench	`rtl/full_adder.v`, `rtl/mult_array.v`, `tb/tb_mult.v`